

`cv2.imread()`

Reads an image from a file.

Syntax: `cv2.imread(filename, flags)`

Parameters:

- `filename` — Path to the image file (string).
- `flags` — Specifies how the image should be read. Options:
 - `cv2.IMREAD_COLOR` (1) — Loads a color image, ignoring any alpha channel. This is the default.
 - `cv2.IMREAD_GRAYSCALE` (0) — Loads image in grayscale mode.
 - `cv2.IMREAD_UNCHANGED` (-1) — Loads image as-is, including the alpha channel.

Code Example (from manual):

`image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)`

`cv2.imshow()`

Displays an image in a window.

Syntax: `cv2.imshow(winname, mat)`

Parameters:

- `winname` — Window name (string). This is the title of the display window.
- `mat` — The image array (NumPy array) to display. The window automatically fits to the image size.

Code Example (from manual):

`cv2.imshow('Segmented Image', segmented_image)`

`cv2.waitKey()`

Waits for a keyboard event.

Syntax: `cv2.waitKey(delay)`

Parameters:

- `delay` — Time in milliseconds to wait for a key press.
 - `delay = 0` → Waits indefinitely until any key is pressed.
 - `delay > 0` → Waits for that many milliseconds (e.g., 5000 = 5 seconds).

Code Example (from manual):

`cv2.waitKey(0) # Wait indefinitely for a key press`

`cv2.destroyAllWindows()`

Closes all OpenCV windows.

Syntax: `cv2.destroyAllWindows()`

Parameters:

- None — This function takes no arguments and returns nothing. It destroys all HighGUI windows created during the program.

Code Example (from manual):

`cv2.destroyAllWindows()`

Thresholding Functions

`cv2.threshold()`

Applies a fixed-level threshold to each pixel.

Syntax: `retval, dst = cv2.threshold(src, thresh, maxval, type)`

Parameters:

- `src` — Input image (must be grayscale, single-channel).
- `thresh` — Threshold value used to classify pixel values.
- `maxval` — Maximum value assigned to pixels that meet the threshold condition. Usually 255.
- `type` — Thresholding type. Options:
 - `cv2.THRESH_BINARY` — If `pixel > thresh` → `maxval`; otherwise → 0.
 - `cv2.THRESH_BINARY_INV` — Opposite of binary: if `pixel > thresh` → 0; otherwise → `maxval`.
 - `cv2.THRESH_TRUNC` — If `pixel > thresh` → `thresh`; otherwise unchanged.
 - `cv2.THRESH_TOZERO` — If `pixel > thresh` → unchanged; otherwise → 0.
 - `cv2.THRESH_TOZERO_INV` — If `pixel > thresh` → 0; otherwise unchanged.
 - `cv2.THRESH_OTSU` — Use Otsu's algorithm to automatically find optimal threshold. Must be combined with one of the above types (e.g., `cv2.THRESH_BINARY + cv2.THRESH_OTSU`). Input must be single-channel.

Returns:

- `retval` — The threshold value used (useful when Otsu is used).
- `dst` — Output binary/thresholded image.

Code Examples (from manual):

Global thresholding

```
ret, binary_mask = cv2.threshold(image, 128, 255, cv2.THRESH_BINARY)
```

Otsu's thresholding

```
ret, binary_mask = cv2.threshold(image, 0, 255, cv2.THRESH_BINARY +  
cv2.THRESH_OTSU)
```

`cv2.adaptiveThreshold()`

Applies adaptive thresholding where the threshold value is computed for each pixel based on its neighborhood.

Syntax: `dst = cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)`

Parameters:

- `src` — Source 8-bit single-channel image.
- `maxValue` — Non-zero value assigned to pixels that satisfy the condition.
- `adaptiveMethod` — Adaptive thresholding algorithm. Options:
 - `cv2.ADAPTIVE_THRESH_MEAN_C` — Threshold is the mean of the `blockSize × blockSize` neighborhood minus `C`.
 - `cv2.ADAPTIVE_THRESH_GAUSSIAN_C` — Threshold is the Gaussian-weighted sum of the neighborhood minus `C`.
- `thresholdType` — Either `cv2.THRESH_BINARY` or `cv2.THRESH_BINARY_INV`.
- `blockSize` — Size of the pixel neighborhood used to calculate the threshold (odd number, e.g., 3, 5, 7, ...).
- `C` — Constant subtracted from the mean or weighted mean. Can be positive or negative.

Returns:

- `dst` — Output thresholded image.
-

Color Conversion & Masking Functions

`cv2.cvtColor()`

Converts an image from one color space to another.

Syntax: `dst = cv2.cvtColor(src, code)`

Parameters:

- `src` — Input image: 8-bit unsigned, 16-bit unsigned, or single-precision floating-point.
- `code` — Color space conversion code. Common options:
 - `cv2.COLOR_BGR2GRAY` — Convert BGR to grayscale.
 - `cv2.COLOR_BGR2RGB` — Convert BGR to RGB.

- cv2.COLOR_BGR2HSV — Convert BGR to HSV.
- cv2.COLOR_RGB2BGR — Convert RGB to BGR.
- cv2.COLOR_BGR2LAB — Convert BGR to LAB.
- cv2.COLOR_BGR2YCrCb — Convert BGR to YCrCb.
- cv2.COLOR_GRAY2BGR — Convert grayscale to BGR.
- dstCn (optional) — Number of channels in the destination image. If 0 (default), it is derived automatically from src and code.

Code Example (from manual):

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
```

cv2.inRange()

Checks if array elements lie between the elements of two other arrays (or scalars).

Produces a binary mask.

Syntax: dst = cv2.inRange(src, lowerb, upperb)

Parameters:

- src — First input array (image).
- lowerb — Inclusive lower boundary array or scalar. Pixels $\geq$ lowerb are included.
- upperb — Inclusive upper boundary array or scalar. Pixels $\leq$ upperb are included.

Returns:

- dst — Output binary image with white (255) where pixels were in-range and black (0) where out-of-range. Same size as src, type CV_8U (8-bit unsigned).

Code Example (from manual):

```
mask = cv2.inRange(hsv_image, lower_bound, upper_bound)
# lower_bound = np.array([30, 50, 50])
# upper_bound = np.array([60, 255, 255])
```

Morphological Operations

cv2.morphologyEx()

Performs advanced morphological transformations using erosion and dilation as basic operations.

Syntax: dst = cv2.morphologyEx(src, op, kernel, anchor, iterations, borderType, borderValue)

Parameters:

- `src` — Source image. Number of channels can be arbitrary. Depth should be one of `CV_8U`, `CV_16U`, `CV_16S`, `CV_32F`, or `CV_64F`.
- `op` — Type of morphological operation. Options:
 - `cv2.MORPH_OPEN` — Opening: erosion followed by dilation. Removes small objects from foreground.
 - `cv2.MORPH_CLOSE` — Closing: dilation followed by erosion. Fills small holes in foreground.
 - `cv2.MORPH_GRADIENT` — Morphological gradient: difference between dilation and erosion. Highlights edges.
 - `cv2.MORPH_TOPHAT` — Top hat: difference between input and opening. Enhances bright regions.
 - `cv2.MORPH_BLACKHAT` — Black hat: difference between closing and input. Enhances dark regions.
- `kernel` — Structuring element. Created with `np.ones((5,5), np.uint8)` or `cv2.getStructuringElement()`.
- `anchor` (optional) — Position of the anchor within the element. Default is `(-1, -1)` meaning the anchor is at the center.
- `iterations` (optional) — Number of times erosion and dilation are applied. Default is 1. Increasing iterations increases the effect.
- `borderType` (optional) — Pixel extrapolation method. Default is `cv2.BORDER_CONSTANT`. Other options include `cv2.BORDER_REPLICATE`, `cv2.BORDER_REFLECT`.
- `borderValue` (optional) — Border value in case of a constant border. Default is `cv2.morphologyDefaultBorderValue()`.

Code Example (from manual):

```
kernel = np.ones((5, 5), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)
```

`cv2.dilate()`

Applies dilation to an image. Dilates the foreground (makes it larger).

Syntax: `dst = cv2.dilate(src, kernel, anchor, iterations, borderType, borderValue)`

Parameters:

- `src` — Input image. Number of channels can be arbitrary. Depth should be `CV_8U`, `CV_16U`, `CV_16S`, `CV_32F`, or `CV_64F`.
- `kernel` — Structuring element used for dilation.
- `anchor` (optional) — Position of the anchor within the element. Default `(-1, -1)` = center.
- `iterations` (optional) — Number of times dilation is applied. Default is 1.

- `borderType` (optional) — Pixel extrapolation method. Default `cv2.BORDER_CONSTANT`. Options: `cv2.BORDER_REPLICATE`, `cv2.BORDER_REFLECT`, etc.
- `borderValue` (optional) — Border value in case of a constant border.

Code Example (from manual):

`sure_bg = cv2.dilate(opening, kernel, iterations=3)`

Distance Transform & Connected Components

`cv2.distanceTransform()`

Calculates the distance from each pixel to the nearest zero pixel.

Syntax: `dst = cv2.distanceTransform(src, distanceType, maskSize, dstType)`

Parameters:

- `src` — 8-bit, single-channel (binary) source image.
- `distanceType` — Type of distance. Options:
 - `cv2.DIST_L1` — Manhattan distance: $|x_1 - x_2| + |y_1 - y_2|$.
 - `cv2.DIST_L2` — Euclidean distance: $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Most commonly used.
 - `cv2.DIST_C` — Chebyshev distance: $\max(|x_1 - x_2|, |y_1 - y_2|)$.
 - `cv2.DIST_L12` — L1-L2 metric.
 - `cv2.DIST_FAIR`, `cv2.DIST_WELSCH`, `cv2.DIST_HUBER` — Other distance metrics.
- `maskSize` — Size of the distance transform mask. Options:
 - `cv2.DIST_MASK_3` (3×3)
 - `cv2.DIST_MASK_5` (5×5)
 - `cv2.DIST_MASK_PRECISE` — Precise (only for `DIST_L2`; not supported for labels variant).
- `dstType` (optional) — Type of output image. Can be `CV_8U` or `CV_32F` (default). `CV_8U` can only be used with `DIST_L1`.

Returns:

- `dst` — Output image with calculated distances. Same size as `src`, type `CV_32F` (or `CV_8U` if specified).

Code Example (from manual):

`dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)`

`cv2.connectedComponents()`

Computes the connected components labeled image of a binary image.

Syntax: `retval, labels = cv2.connectedComponents(image, connectivity, ltype)`

Parameters:

- `image` — 8-bit single-channel binary image.
- `connectivity` (optional) — Connectivity to use for the labeling procedure. Options:
 - 4 — 4-way connectivity (only horizontal and vertical neighbors).
 - 8 — 8-way connectivity (includes diagonal neighbors). This is the default.
- `ltype` (optional) — Output image label type. Options: `cv2.CV_32S` (default), `cv2.CV_16U` (requires `connectivity=8`).

Returns:

- `retval` — Number of labels (components found).
- `labels` — Labeled image where each connected component has a unique integer label.

Code Example (from manual):

```
_, markers = cv2.connectedComponents(sure_fg)  
markers = markers + 1
```

Arithmetic Operations

`cv2.subtract()`

Calculates the per-element difference between two arrays or an array and a scalar.

Syntax: `dst = cv2.subtract(src1, src2, mask, dtype)`

Parameters:

- `src1` — First input array or scalar.
- `src2` — Second input array or scalar. Must have the same size and number of channels as `src1`.
- `mask` (optional) — Operation mask (8-bit single-channel array). Specifies elements of the output array to change. Default is `noArray()` (all elements are changed).
- `dtype` (optional) — Optional depth of the output array. Default is -1 (same depth as input).

Returns:

- `dst` — Output array of the same size and number of channels as the input arrays.

Code Example (from manual):

```
unknown = cv2.subtract(sure_bg, sure_fg)
```

Edge Detection

`cv2.Canny()`

Finds edges in an image using the Canny algorithm.

Syntax: `edges = cv2.Canny(image, threshold1, threshold2, apertureSize, L2gradient)`

Parameters:

- `image` — Single-channel 8-bit input image.
- `threshold1` — First threshold for the hysteresis procedure (lower threshold).
- `threshold2` — Second threshold for the hysteresis procedure (higher threshold).
- `apertureSize` (optional) — Aperture size for the Sobel operator. Default is 3. Options: 3, 5, 7 (must be odd).
- `L2gradient` (optional) — Flag indicating whether a more accurate L2 norm should be used to calculate the image gradient magnitude. Default is `False` (uses L1 norm).

Returns:

- `edges` — Output edge map. Single-channel 8-bit image with the same size as the input. Edges are white (255), non-edges are black (0).

Code Example (from manual):

`edges = cv2.Canny(image, 100, 200)`

Watershed Segmentation

`cv2.watershed()`

Performs a marker-based image segmentation using the watershed algorithm.

Syntax: `markers = cv2.watershed(image, markers)`

Parameters:

- `image` — Input 8-bit 3-channel image.
- `markers` — Input/output 32-bit single-channel image (map) of markers. Must have the same size as `image`.
 - Input markers: Positive values (> 0) indicate known regions (markers); 0 indicates unknown region.
 - Output markers: Boundaries are marked with -1. Each region gets a unique positive integer label.

Returns:

- `markers` — Updated marker image where boundaries between regions are marked as -1.

Code Example (from manual):

cv2.watershed(image, markers)

image[markers == -1] = [255, 0, 0] # Mark boundaries with red

Clustering

cv2.kmeans()

Finds centers of clusters and groups input samples around the clusters.

Syntax: compactness, labels, centers = cv2.kmeans(data, K, bestLabels, criteria, attempts, flags)

Parameters:

- data — Data for clustering. Floating-point matrix with one row per sample, each column a feature. Must be np.float32.
- K — Number of clusters to split the set by (integer).
- bestLabels — Input/output integer array storing cluster indices for each sample. Can be None if not needed as input.
- criteria — Algorithm termination criteria. A tuple of 3 parameters: (type, max_iter, epsilon):
 - type — Termination criteria type. Options:
 - cv2.TERM_CRITERIA_EPS — Stop when specified accuracy epsilon is reached.
 - cv2.TERM_CRITERIA_MAX_ITER — Stop after max_iter iterations.
 - cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER — Stop when either condition is met.
 - max_iter — Maximum number of iterations (integer).
 - epsilon — Desired accuracy (float).
- attempts — Number of times the algorithm is executed with different initial labellings. The algorithm returns the best result (based on compactness).
- flags — Flag to specify how initial centers are chosen. Options:
 - cv2.KMEANS_RANDOM_CENTERS — Random initial centers are selected in each attempt.
 - cv2.KMEANS_PP_CENTERS — Initial centers are chosen using k-means++ algorithm.

Returns:

- compactness — Sum of squared distance from each point to its corresponding center.
- labels — Label array where each element is labeled 0, 1, etc. (same as bestLabels output).
- centers — Array of cluster centers.

Code Example (from manual):

```
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)  
K = 4  
_, labels, centers = cv2.kmeans(pixel_values, K, None, criteria, 10,  
cv2.KMEANS_RANDOM_CENTERS)
```

Quick Reference Table

Function	Key Parameters	Important Options
cv2.imread	filename, flags	IMREAD_COLOR, IMREAD_GRAYSCALE, IMREAD_UNCHANGED
cv2.imshow	winname, mat	—
cv2.waitKey	delay	0 = infinite, >0 = milliseconds
cv2.threshold	src, thresh, maxval, type	THRESH_BINARY, THRESH_BINARY_INV, THRESH_TRUNC, THRESH_TOZERO, THRESH_TOZERO_INV, THRESH_OTSU
cv2.adaptiveThreshold	src, maxValue, adaptiveMethod, thresholdType, blockSize, C	ADAPTIVE_THRESH_MEAN_C, ADAPTIVE_THRESH_GAUSSIAN_C
cv2.cvtColor	src, code, dstCn	COLOR_BGR2GRAY, COLOR_BGR2HSV, COLOR_BGR2RGB, COLOR_BGR2LAB, etc.

cv2.inRange	src, lowerb, upperb	Lower/upper bounds as arrays or scalars
cv2.morphologyEx	src, op, kernel, anchor, iterations	MORPH_OPEN, MORPH_CLOSE, MORPH_GRADIENT, MORPH_TOPHAT, MORPH_BLACKHAT
cv2.dilate	src, kernel, anchor, iterations	—
cv2.distanceTransform	src, distanceType, maskSize, dstType	DIST_L1, DIST_L2, DIST_C; DIST_MASK_3, DIST_MASK_5, DIST_MASK_PRECISE
cv2.connectedComponents	image, connectivity, ltype	connectivity: 4 or 8
cv2.subtract	src1, src2, mask, dtype	mask optional, dtype optional
cv2.Canny	image, threshold1, threshold2, apertureSize, L2gradient	apertureSize: 3, 5, 7; L2gradient: True/False
cv2.watershed	image, markers	markers: input/output, boundaries = -1
cv2.kmeans	data, K, bestLabels, criteria, attempts, flags	TERM_CRITERIA_EPS, TERM_CRITERIA_MAX_ITER; KMEANS_RANDOM_CENTERS, KMEANS_PP_CENTERS

Exam Tips

- Know the parameter names and their order for each function.

- Memorize the threshold types (THRESH_BINARY, THRESH_OTSU, etc.) — these are frequently tested.
- Understand the return values — especially cv2.threshold returns two values (retval, dst), and cv2.kmeans returns three (compactness, labels, centers).
- Distinguish between cv2.threshold and cv2.adaptiveThreshold — the latter uses blockSize and C instead of a single thresh.
- Remember the morphological operations and what each does (MORPH_OPEN removes noise, MORPH_CLOSE fills holes, etc.).
- For cv2.watershed, know that markers are required and boundaries are marked as -1 in the output.